

Linear and Logistic Regression

Course Facilitators

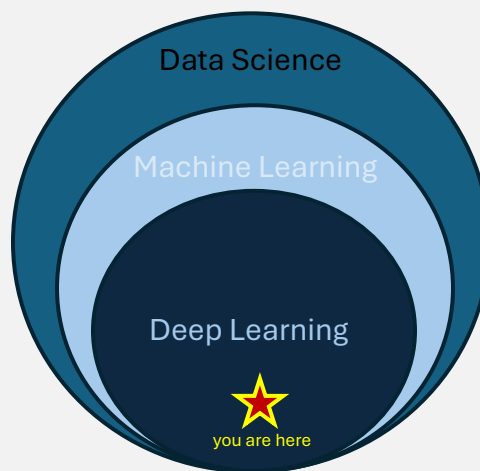
- **Course Instructor:** [Jonathan Schlosser](#), Adjunct Professor
 - jschloss@unc.edu
 - **Office Hours: Mondays 5PM – 6PM**
 - PhD in Media and Communication from Chapel Hill
 - Lead Data Scientist – Generative AI, AlignAI
 - Research Interests
 - LLMs, MLOps, and Generative AI
 - Applied AI Development
 - Generative Documentation Systems
 - Recent Projects
 - <https://www.jonathanschlosser.com/projects/formbot>
 - <https://www.jonathanschlosser.com/projects/automated-pipeline-monitoring>
- **Course Designer:** [Shashank Srivastava](#), Asst. Professor
 - ssrivastava@cs.unc.edu
 - PhD in Machine Learning from Carnegie Mellon University
 - Leads the “Learning from Language Lab” at UNC Chapel Hill
 - Research Interests
 - Language, intelligence, and Interactivity
 - Explanation & Alignment
 - Social & Cognitive Applications
 - Recent Publications
 - Is Chain-of-Thought Really Not Explainability? Chain-of-Thought Can Be Faithful without Hint Verbalization
 - <https://arxiv.org/abs/2512.23032>

- **DiffVax: Optimization-Free Image Immunization Against Diffusion-Based Editing**
 - <https://diffvax.github.io/>

Course Overview

- **What is Deep Learning?**

- a subset of machine learning that has to do with building machine learning models that are large, complex, and differentiable



- **Applications of Deep Learning**

- **Healthcare:** from early disease detection to robotic surgeries
- **Transport and automation:** powering autonomy in self-driving cars
- **Entertainment:** personalizing streaming content
- **Science:** predicting protein structures, creating new molecules
- **Technology and information:** ChatGPT, Bard, and other language technologies

- **Impact on Daily Life**

- **Facial recognition** in your phones
- **Speech technology** in Siri and Alexa

	○ Automated customer service and chatbots ○ Real-time translation apps ○ Home automation and the Internet of Things (IoT) ○ Fraud detection with credit cards ● Breakthroughs and Innovations ○ AlphaGo defeats human champions (2016) ○ Insilico Medicine identifies a new molecule for accelerated treatment of fibrosis in 46 days (2019) ○ Discover of exoplanets: Kepler-90i and Kepler-80g (2017) ○ Early detection of diabetic retinopathy (2021)
What to Expect	 not  images: ChatGPT 5.4 ● heavy emphasis on linear algebra and calculus ● drudgery and hard work ● principles and methods underlying modern tools, but not the tools or libraries themselves ● “unless we grossly mess it up, this course is going to be hard”
Unit 1 Overview	● distinguish between traditional machine learning pipelines and deep learning models ● design linear models for predictive modeling ● implement gradient descent to train linear regression ● understand the classification approach presented in logistic regression

Async Materials

Linear Regression

Week 1 Outline

- **form and model parameters**
- **loss function**
- **training with gradient descent**
- **mini-batch stochastic gradient descent**
- **hyper-parameters**

Form and Model Parameters

Regression

- **predicting a real-value (scalar) output based on a set of input features**

$$f: \mathbf{x} \in \mathbb{R}^d \rightarrow y \in \mathbb{R}$$

- e.g. **estimate the energy consumption of a home from**
 - **size**: floorspace (*sq ft*)
 - **temp**: avg temperature (*F*)
 - **prev**: consumption in previous month (*Kw*)

Linear Regression

- when the **function f** is **linear** in the **inputs $\mathbf{x}$**

$$\begin{aligned} y: & \text{consumption} \\ x: & [\text{size}, \text{temp}, \text{prev}] \\ w: & [w_1, w_2, w_3] \end{aligned}$$

- that is: **consumption** = $w_1 \text{size} + w_2 \text{temp} + w_3 \text{previous} + b$

$$y = \mathbf{w}^T \mathbf{x} + b$$

- training set:

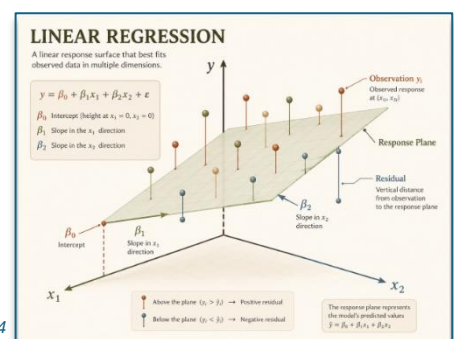
$$\mathbf{X}_{\text{train}}, \mathbf{Y}_{\text{train}} := \{(\mathbf{x}^k, y^k)_n\}$$

$$\hat{\mathbf{Y}} = \mathbf{X}\mathbf{w} + \mathbf{b}$$

$n \times d$ $d \times 1$ $n \times 1$

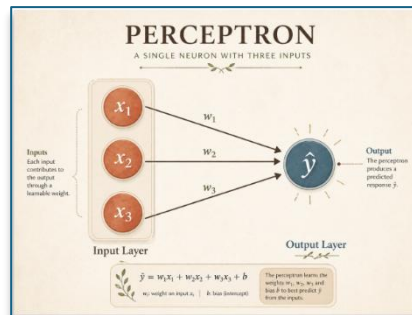
scalar (bias)

ChatGPT 5.4



Linear Regression as a Neural Net

- linear regression can be thought of as a **single-layer neural net** (also called a **perceptron**)



$$\hat{y} = w^T x$$

Loss Function

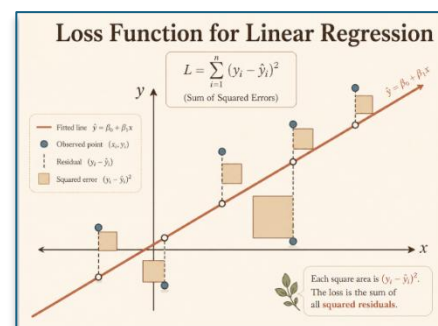
Loss Function for Linear Regression

- describes “how well” your model is performing
- loss function:

$$L(w) := \frac{1}{2} \sum_k (y^k - \hat{y}^k)^2$$

$$= L(w) := \frac{1}{2} \sum_k (y^k - w^T x^k)^2$$

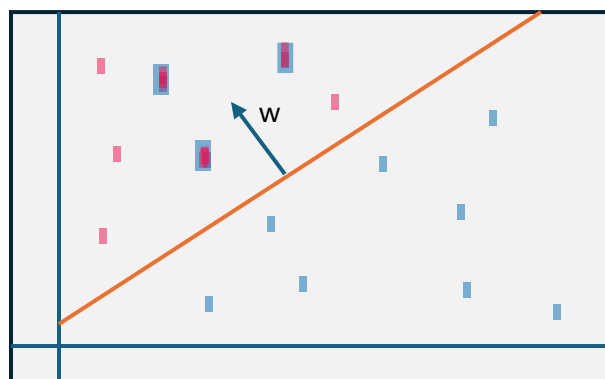
$$= \text{sum}(\text{squared errors})$$



Training with Gradient Descent

Inference / Testing

- the classifier is a hyperplane specified by parameters (w, b)
- test: which side of the hyperplane does the test instance fall?



Model Training / Parameter Estimation	- maximize the conditional likelihood of the labeled data - learned parameters are maximum likelihood estimates (MLE) $W_{MLE} = \underset{W}{\operatorname{argmax}} P(Y X, W)$ - parameters learned through gradient descent ascent - gradient: prediction error $\frac{\partial \log P(Y X, W)}{\partial w_i} = \sum_l X_i^l (Y_l - \underbrace{P(Y^l = 1 X^l, W)})$
Regularization	
Overfitting	- can happen with too many features and too few examples overfitting is bad - you might be able to explain every training example (100% training accuracy) - this will not generalize (poor test performance) e.g., spam vs. not-spam classification - too few train instances - overfitting - rule="IF containsWord("\$999") THEN label=spam" - many train instances - good! - rule="IF containsMoney() THEN label=spam" []
Underfitting	- happens when you don't learn well enough - e.g., rule: "always predict label=spam" - doesn't generalize (poor test performance)

w to avoid

- underfitting
 - check performance on training set
- overfitting
 - check performance on validation set
 - use regularization

Regularization

- add a penalty term to the maximization objective function
 - old optimization

$$W_{MLE} = \operatorname{argmax}_W P(Y|X, W)$$

- new optimization

$$W_{MLE} = \operatorname{argmax}_W P(Y|X, W) - \lambda R(W)$$

- penalize high weights in w

- L2 regularization
 - sum of squared values of weights
 - L1 regularization
 - sum of absolute values of weights



- how to choose λ ?
 - is a hyperparameter so will not be optimized
 - most common method is to check performance on held-out validation set

